

datalex



Use Delta Lake





Alexandre BERGERE

Head of Data & AI Engineer & Partners at DataGalaxy, Data Architect independent
Delta & openLineage lover

Data Expert
(Audit, Modeling, Urbanization, BI's project – Azure, Big Data, Spark, openLineage & Delta lover)

Trainer
(BI, Big Data, MongoDB, NoSQL, SQL, Cloud Computing)

Speaker
(Pass Summit 2018, Spark AI Tour 2020, …)

Cloud & data tech articles
(medium, LinkedIn, slideshare)

Avanade
Sr Anls, Data Analyst

2016 - 2019

Trainer
2016 - x

ESAIP, ESME, ECS, NEOMA, ESEO, EFREI, Sorbonne Paris

BI, Big Data, MongoDB, NoSQL, Panorama de la data, Cloud computing

OpenClassroom
Mentor

2019 - x

Datalex
2019 - x

Data Architect Freelance

DataRedKite
CTO & Co-Founder

2020 - 2023

DataGalaxy
2021 - x

Head of Data & AI Engineer

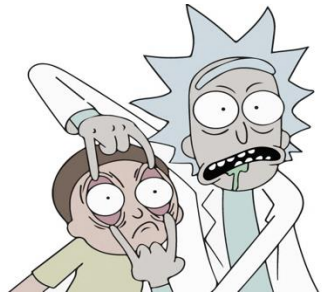
Don’t get lost



Practice



Demo

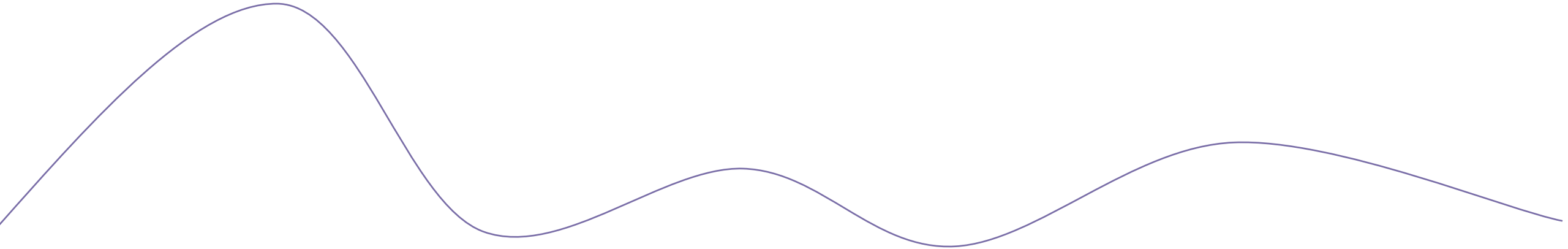


Important

Learning Objectives

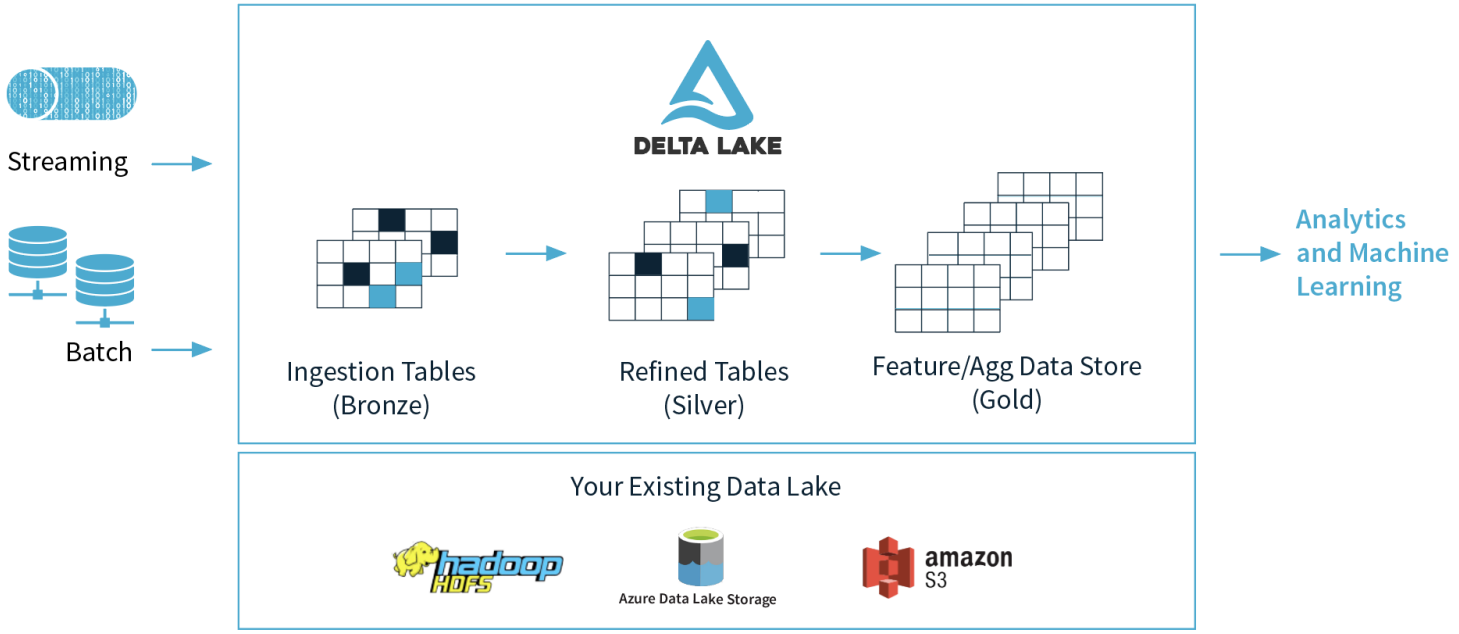
1. Understand Delta Lake
2. Set Up Delta Tables
3. Load Data into Delta Lake
4. Version and Optimize Delta Tables

Delta Lake



Delta Lake

Delta Lake is an open-source storage layer that brings ACID transactions (~adds relational database semantics) to Apache Spark™ and big data workloads.



Delta Lake uses versioned Parquet files to store your data in your cloud storage. Apart from the versions, Delta Lake also stores a transaction log to keep track of all the commits made to the table or blob store directory to provide ACID transactions.

Delta Lake brings ACID to object storage

- **Atomicity** means all transactions either succeed or fail completely.
- **Consistency** guarantees relate to how a given state of the data is observed by simultaneous operations.
- **Isolation** refers to how simultaneous operations conflict with one another. The isolation guarantees that Delta Lake provides do differ from other systems.
- **Durability** means that committed changes are permanent.

Problems solved by ACID

- Hard to append data
- Modification of existing data difficult
- Jobs failing mid way
- Real-time operations hard
- Costly to keep historical data versions

Delta Lake's Key Features

- Relational tables that support querying and data modification
- Support for ACID transactions
- Data versioning and time travel
- Schema enforcement and evolution
- Audit history
- Support for batch and streaming data
- Standard formats and interoperability
- Compatible with Apache Spark API

Delta Lake

Key features (1/2)

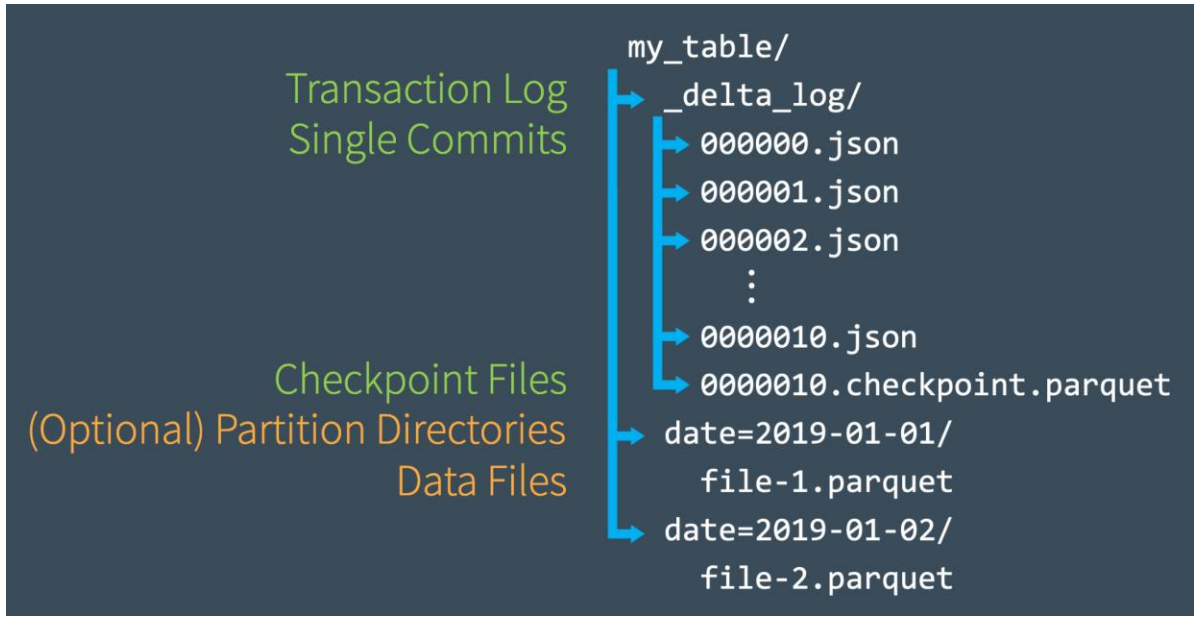
Feature	Description
ACID Transactions	Data lakes typically have multiple data pipelines reading and writing data concurrently, and data engineers have to go through a tedious process to ensure data integrity, due to the lack of transactions. Delta Lake brings ACID transactions to your data lakes. It provides serializability, the strongest level of isolation level.
Scalable Metadata Handling	In big data, even the metadata itself can be “big data”. Delta Lake treats metadata just like data, leveraging Spark’s distributed processing power to handle all its metadata. As a result, Delta Lake can handle petabyte-scale tables with billions of partitions and files at ease.
Time Travel (data versioning)	Delta Lake provides snapshots of data enabling developers to access and revert to earlier versions of data for audits, rollbacks or to reproduce experiments.
Open Format	All data in Delta Lake is stored in Apache Parquet format enabling Delta Lake to leverage the efficient compression and encoding schemes that are native to Parquet.
Unified Batch and Streaming Source and Sink	A table in Delta Lake is both a batch table, as well as a streaming source and sink. Streaming data ingest, batch historic backfill, and interactive queries all just work out of the box.

Delta Lake

Key features (1/2)

Feature	Description
Schema Enforcement	Delta Lake provides the ability to specify your schema and enforce it. This helps ensure that the data types are correct and required columns are present, preventing bad data from causing data corruption.
Schema Evolution	Big data is continuously changing. Delta Lake enables you to make changes to a table schema that can be applied automatically, without the need for cumbersome DDL.
100% Compatible with Apache Spark API	Developers can use Delta Lake with their existing data pipelines with minimal change as it is fully compatible with Spark, the commonly used big data processing engine.
Audit History	Delta Lake transaction log records details about every change made to data providing a full audit trail of the changes.
Full DML Support	Delta Lake supports standard DML including UPDATE, DELETE and MERGE INTO providing developers more controls to manage their big datasets.

Transaction logs



https://databricks.com/fr/blog/2019/08/21/diving-into-delta-lake-unpacking-the-transaction-log.html

Delta Lake implementation

Instead of **parquet**…

```
dataframe
  .write
  .format("parquet")
  .save("/data")
```

…simply say **delta**

```
dataframe
  .write
  .format("delta")
  .save("/data")
```

Delta Lake on Databricks

Delta Lake is the default format for tables created in Databricks.

```
CREATE TABLE foo
~~USING DELTA~~
```

```
df.write~~.format("delta")~~
```

Create Delta Lake tables

Create a Delta Lake table from a dataframe

```
df = spark.read.load("/data/mydata.csv", format="csv", header=True)
delta_table_path = "/delta/mydata"
df.write.format("delta").save(delta_table_path)
```

Make conditional updates

```
from delta.tables import *
from pyspark.sql.functions import *

deltaTable = DeltaTable.forPath(spark, delta_table_path)
deltaTable.update(
condition = "Category == 'Accessories'",
set = { "Price": "Price * 0.9" })
```

Query a previous version *(time travel)*

```
df = spark.read.format("delta").option("versionAsOf", 0).load(delta_table_path)
```

Create catalog tables

Managed tables

- Defined without a specific location – files are created in metastore folder
- Dropping the table deletes the files

```
df.write.format("delta").option("path","/mydata").saveAsTable("MyExternalTable")
```

External tables

- Defined with a specific file location
- Dropping the table does not delete the files

```
spark.sql("CREATE TABLE MyExternalTable USING DELTA LOCATION '/mydata'")
```

```
%%sql
CREATE TABLE MyExternalTable
USING DELTA
LOCATION '/mydata'
```

Use Delta Lake with streaming data

Use Delta Lake table as a streaming source:

```
from pyspark.sql.types import *
from pyspark.sql.functions import *

stream_df = spark.readStream.format("delta") \
.option("ignoreChanges", "true") \
.load("/delta/internetorders")

stream_df.show()
```

Use Delta Lake table as a streaming sink

```
from pyspark.sql.types import *
from pyspark.sql.functions import *

stream_df = spark.readStream.schema(jsonSchema).option("maxFilesPerTrigger", 1).json(inputPath)
table_path = '/delta/devicetable'
checkpoint_path = '/delta/checkpoint'
delta_stream = stream_df.writeStream.format("delta").option("checkpointLocation", checkpoint_path).start(table_path)
```

DE 3.1 - Schemas and Tables

- Use Spark SQL DDL to define schemas and tables
- Describe how the LOCATION keyword impacts the default storage directory



DE 3.2 - Set Up Delta Tables

- Use CTAS statements to create Delta Lake tables
- Create new tables from existing views or tables
- Enrich loaded data with additional metadata
- Declare table schema with generated columns and descriptive comments
- Set advanced options to control data location, quality enforcement, and partitioning
- Create shallow and deep clones



DE 3.3 - Load Data into Delta Lake

- Overwrite data tables using INSERT OVERWRITE
- Append to a table using INSERT INTO
- Append, update, and delete from a table using MERGE INTO
- Ingest data incrementally into tables using COPY INTO



DE 3.4L - Load Data Lab



DE 3.5 - Version and Optimize Delta Tables

- Use OPTIMIZE to compact small files
- Use ZORDER to index tables
- Describe the directory structure of Delta Lake files
- Review a history of table transactions
- Query and roll back to previous table version
- Clean up stale data files with VACUUM

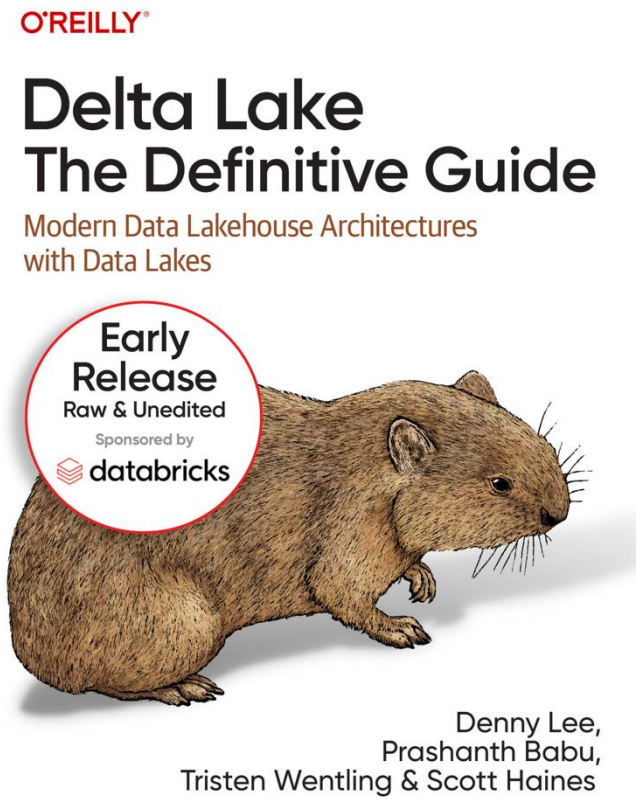


DE 3.6L - Manipulate Delta Tables Lab



Lake: The Definitive Guide

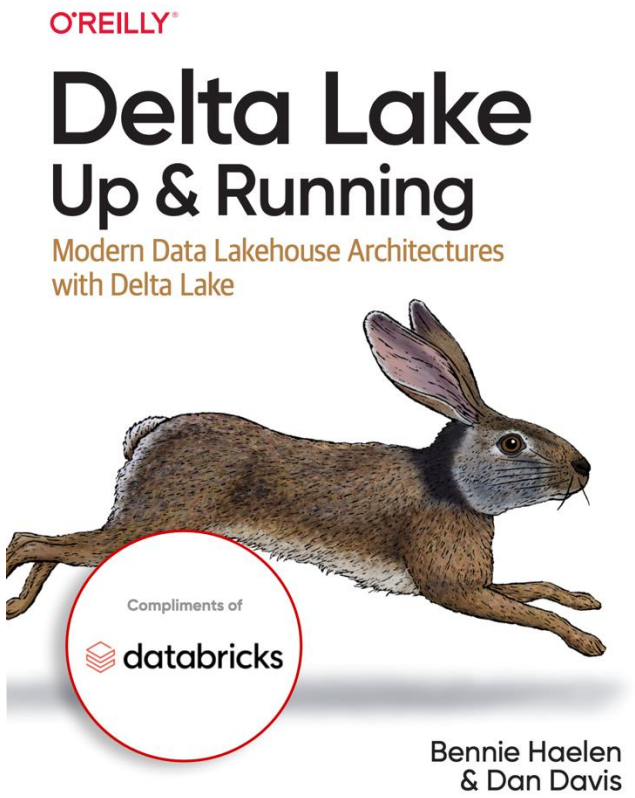




Free pdf link

Delta Lake: Up & Running by O'ReillyDelta





Free pdf link